

Aim: Support Vector Machines (SVM)

Implement the SVM algorithm for binary classification.

Train an SVM model using a given dataset and optimize its parameters.

Evaluate the performance of the SVM model on test data and analyze the results.

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

df = pd.read_csv("iris.csv")
df = df.drop(columns=["Id"])
df = df[df["Species"].isin(["Iris-versicolor", "Iris-virginica"])]

X = df.drop(columns=["Species"])
y = df["Species"].map({"Iris-versicolor": 0, "Iris-virginica": 1})

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42, stratify=y)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

param_grid = {
    "C": [0.1, 1, 10, 100],
    "gamma": ["scale", 0.1, 0.01, 0.001],
    "kernel": ["linear", "rbf"]
}

grid = GridSearchCV(SVC(), param_grid, cv=5, scoring="accuracy")
grid.fit(X_train_s, y_train)

print(f"Best Parameters: {grid.best_params_}")
print(f"Best CV Accuracy: {grid.best_score_:.4f}")
```

```
print()

model = grid.best_estimator_
y_pred = model.predict(X_test_s)

acc = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {acc:.4f}")
print()
print("Classification Report:")
print(classification_report(y_test, y_pred,
target_names=["Iris-versicolor", "Iris-virginica"]))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print(f"Support Vectors: {model.n_support_}")

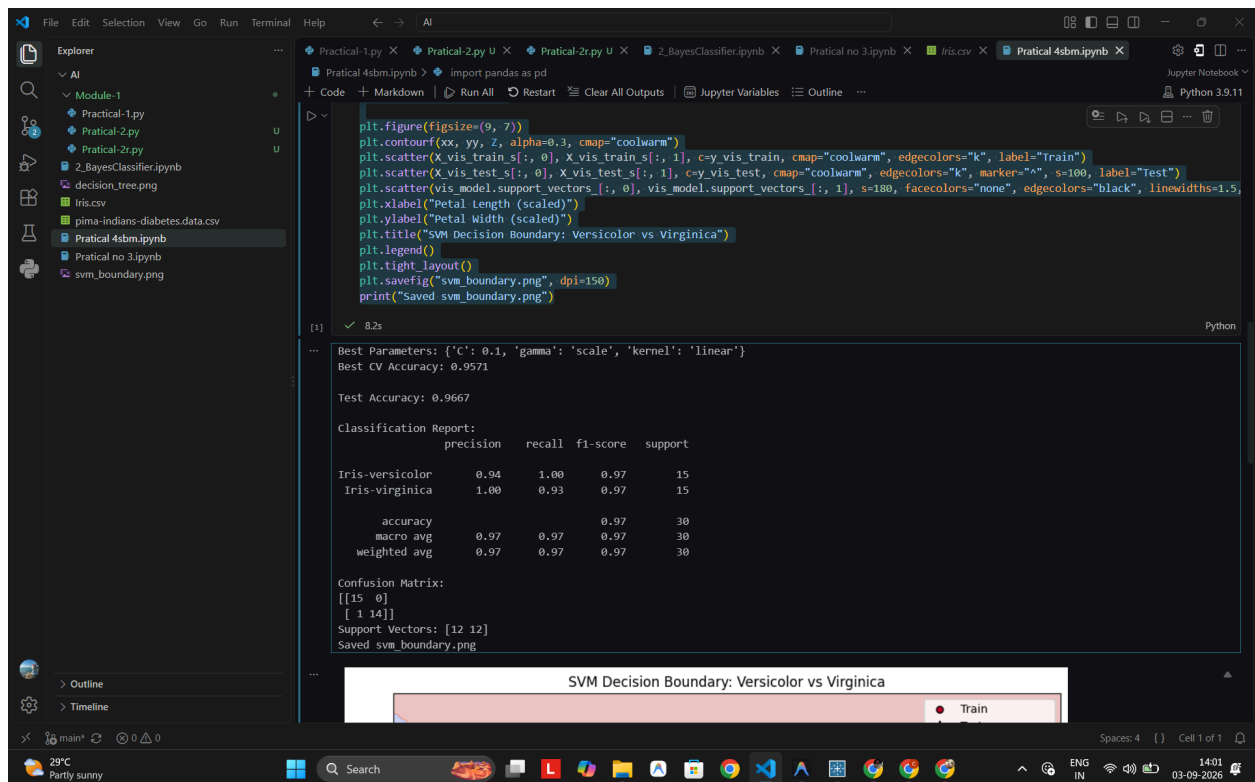
X_vis = X[["PetalLengthCm", "PetalWidthCm"]].values
X_vis_train, X_vis_test, y_vis_train, y_vis_test = train_test_split(X_vis,
y, test_size=0.3, random_state=42, stratify=y)
scaler_vis = StandardScaler()
X_vis_train_s = scaler_vis.fit_transform(X_vis_train)
X_vis_test_s = scaler_vis.transform(X_vis_test)

vis_model = SVC(C=grid.best_params_["C"],
gamma=grid.best_params_["gamma"], kernel=grid.best_params_["kernel"])
vis_model.fit(X_vis_train_s, y_vis_train)

x_min, x_max = X_vis_train_s[:, 0].min() - 1, X_vis_train_s[:, 0].max() +
1
y_min, y_max = X_vis_train_s[:, 1].min() - 1, X_vis_train_s[:, 1].max() +
1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), np.arange(y_min,
y_max, 0.02))
Z = vis_model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(9, 7))
plt.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
plt.scatter(X_vis_train_s[:, 0], X_vis_train_s[:, 1], c=y_vis_train,
cmap="coolwarm", edgecolors="k", label="Train")
```

```
plt.scatter(X_vis_test_s[:, 0], X_vis_test_s[:, 1], c=y_vis_test,
cmap="coolwarm", edgecolors="k", marker="^", s=100, label="Test")
plt.scatter(vis_model.support_vectors[:, 0],
vis_model.support_vectors[:, 1], s=180, facecolors="none",
edgecolors="black", linewidths=1.5, label="Support Vectors")
plt.xlabel("Petal Length (scaled)")
plt.ylabel("Petal Width (scaled)")
plt.title("SVM Decision Boundary: Versicolor vs Virginica")
plt.legend()
plt.tight_layout()
plt.savefig("svm_boundary.png", dpi=150)
print("Saved svm_boundary.png")
```



Sheth L.U.J & Sir M.V College
Subject:AI

